L22 - 面试通关指南

> **"把项目写进简历，拿下Offer"**

本节目标

1. 学会在简历中精准描述 MiniMind 项目
2. 准备好 30秒/1分钟/3分钟 的项目介绍
3. 掌握面试官最常问的 Top 20 问题的标准答案
4. 学会应对深挖追问
5. 了解加分项和面试技巧

前置知识

- 完成 L01-L21 的所有课程学习
- 对 MiniMind 的整体架构和训练流程有清晰认知
- 实际动手跑过训练或推理（强烈建议）

一、简历中如何描述 MiniMind 项目

1.1 STAR 法则

简历描述项目应遵循 STAR 法则：

要素	含义	对应 MiniMind
Situation	背景	大模型技术快速发展，但大部分项目门槛高
Task	任务	从零实现一个完整的小型 LLM
Action	行动	实现了 Transformer 架构、预训练、SFT、DPO/PPO、MoE 等
Result	结果	64M 参数模型，支持多轮对话、Tool Calling 等

1.2 简历描述示例

简洁版（适合简历项目经历，2-3 行）

```
MiniMind - 从零实现的小型大语言模型                                2024.xx - 2024.xx
• 基于 PyTorch 从零实现 64M 参数的 Decoder-Only Transformer，完成预训练、SFT、
  DPO/RLHF 全流程训练，支持多轮对话和工具调用
• 实现 RoPE、GQA、SwiGLU、RMSNorm、KV-Cache 等核心技术，
  并扩展 MoE 架构（198M/A64M，4 experts/top-1）
```

标准版（适合详细简历或项目说明，4-5 行）

```
MiniMind - 全流程大语言模型实践                                    2024.xx - 2024.xx
• 基于 PyTorch 从零实现 Decoder-Only Transformer (64M 参数)，采用 RoPE 位置编码、
  GQA 注意力机制、SwiGLU 激活函数、RMSNorm 归一化等主流 LLM 技术
• 完成预训练(3B tokens)→SFT(多轮对话)→DPO(偏好对齐)→PPO/GRPO(强化学习)全链路训练
• 实现 MoE 混合专家扩展版本(4 experts/top-1, 198M/A64M)，支持 KV-Cache 推理优化、
  YaRN 长度外推(512K tokens)
• 部署兼容 OpenAI API 接口，支持 ollama/vLLM/llama.cpp 推理，
  实现 Tool Calling 和 Adaptive Thinking 功能
```

详细版（适合项目文档或面试准备材料，6-8 行）

```
MiniMind - 端到端大语言模型全流程实践                              2024.xx - 2024.xx
• 基于 PyTorch 从零构建 Decoder-Only Transformer 架构 (64M 参数, 8 层, 512 维隐藏层)，
  集成 RoPE 旋转位置编码、GQA 分组查询注意力(8Q/2KV)、SwiGLU 激活函数、RMSNorm 归一化
• 构建自定义 BPE 分词器(词表 6400)，基于 3B tokens 中文语料完成预训练，
  使用 LoRA(r=16) 进行高效参数微调
• 实现 SFT 多轮对话微调(Loss Mask 机制)、DPO 偏好对齐、PPO/GRPO 强化学习训练，
  使用 InternLM2-1.8B-Reward 作为奖励模型
• 扩展 MoE 混合专家架构(4 experts/top-1, 198M 总参/64M 激活参)，
  实现负载均衡 Loss 防止专家塌缩
• 实现 KV-Cache 推理加速、YaRN RoPE 长度外推(32K→512K)、
  多种解码策略(Top-K/Top-P/Temperature/Repetition Penalty)
• 部署 OpenAI 兼容 API，支持 ollama/vLLM/llama.cpp 推理框架，
  实现 Tool Calling (<tool_call>标签) 和 Adaptive Thinking (<think>标签) 功能
```

1.3 简历描述注意事项

1. **量化**：尽量用数字说话（64M 参数、3B tokens、4 experts 等）
2. **技术关键词**：面试官搜简历时会用关键词匹配，确保覆盖核心技术名称
3. **层次感**：从架构 → 训练 → 优化 → 部署，展示完整能力链
4. **诚实**：只写你真正理解的部分，不要写"参与 xxx"

二、面试中如何介绍项目

2.1 30 秒版本（电梯演讲）

> "我做了一个叫 MiniMind 的项目，是从零用 PyTorch 实现的小型大语言模型，64M 参数。我完成了从预训练、SFT 微调到 DPO 和 PPO 强化学习的全流程训练，还实现了 MoE 混合专家架构。目前支持多轮对话、工具调用等功能，可以通过 ollama 等框架部署。通过这个项目，我深入理解了 LLM 的核心技术和训练范式。"

2.2 1 分钟版本

> "我的项目叫 MiniMind，是一个从零构建的小型大语言模型。
>
> **架构方面**，我用 PyTorch 实现了完整的 Decoder-Only Transformer，集成了 RoPE 位置编码、GQA 注意力、SwiGLU 激活函数这些主流 LLM 技术，模型只有 64M 参数。
>
> **训练方面**，我走完了 LLM 的全部训练流程：预训练用了约 3B tokens 的中文语料，然后做了 SFT 多轮对话微调，接着用 DPO 做偏好对齐，还用 PPO 和 GRPO 做了强化学习训练。
>
> **扩展方面**，我实现了 MoE 混合专家架构——4 个专家、Top-1 路由，总参数 198M 但激活参数只有 64M。推理上支持 KV-Cache 加速和 YaRN 长度外推。
>
> **部署方面**，模型兼容 OpenAI API，支持 ollama、vLLM、llama.cpp 等推理框架，还实现了 Tool Calling 和自适应思考功能。"

2.3 3 分钟版本

在 1 分钟版本基础上，按面试官的兴趣点展开。**关键原则**：

1. 先给全景，再深入细节
2. 准备好每个技术点的"2 句话解释"
3. 主动引导到你最擅长的方向

> "... [1 分钟版本内容] ...
>
> 我重点讲讲几个有挑战性的部分：

> **RoPE 位置编码**，我研究了它的数学推导——通过旋转矩阵将位置信息编码到 Q 和 K 中，相对位置关系体现在内积中，这比绝对位置编码更灵活。
>
> **DPO 训练**是偏好对齐的部分。DPO 的巧妙之处在于不需要单独训练 Reward Model，直接从人类偏好对数据中学习，数学上等价于一个隐式的 RLHF。
>
> **MoE 架构**的实现要解决负载均衡问题。我用了辅助 Loss 来防止所有 token 涌向同一个专家，这个问题在实际训练中很常见。
>
> 通过这个项目，我对 LLM 从架构设计到训练再到部署有了完整的理解，也积累了实际的调参和排错经验。"

三、面试官最常问的 Top 20 问题及标准答案

Q1: 为什么选择这个项目?

标准答案:

> "我选择这个项目有两个原因。第一，大模型是当前 AI 最核心的方向，我希望不只是调用 API，而是真正理解底层原理；第二，MiniMind 的 64M 参数规模使得我可以在单卡 GPU 上完成全流程训练，这让我能够快速迭代、深入理解每个技术细节。"

深挖追问：你自己改了什么 / 遇到了什么困难 → 转入具体技术问题

Q2: 模型架构是什么?

标准答案:

> "MiniMind 使用 Decoder-Only Transformer 架构。具体来说： - 8 层 Transformer Block - 512 维隐藏层 - 8 个 Query Head，2 个 KV Head（GQA） - SwiGLU 激活函数 - RMSNorm 归一化 - RoPE 旋转位置编码 - 词表大小 6400 - 参数量约 64M"

Q3: 参数量是多少，怎么算的?

标准答案:

> "总参数约 64M。计算方式： - Embedding: vocab_size × dim = 6400 × 512 ≈ 3.3M - 每层 Attention: Q/K/V/O 投影 ≈ dim^2 × 4 (考虑 GQA 后略小) - 每层 FFN: SwiGLU 有 3 个线性层 ≈ dim × hidden_dim × 3 - RMSNorm: 每层 2 个 ≈ 2 × dim - 8 层累加 - LM Head 与 Embedding 共享权重，不额外占参数
>
> 实际计算需要考虑 GQA 减少的 KV 参数和具体的 FFN 隐藏维度。"

Q4: RoPE 是什么，怎么实现的?

标准答案:

> "RoPE（Rotary Position Embedding）通过旋转矩阵编码位置信息。核心思想是给 Q 和 K 向量乘以一个与位置相关的旋转矩阵。
>
> 具体实现：将向量的维度两两配对，每对用一个旋转角度 θ。位置 m 的旋转角度为 mθ_i，其中 θ_i = 10000^{-2i/d}。
>
> 关键性质：两个位置的 Q 和 K 做内积后，旋转角度变成它们的差 (m-n)θ_i，自然编码了相对位置。
>
> 实现上用复数乘法代替矩阵乘法，效率很高。"

Q5: GQA 是什么，和 MHA 的区别?

标准答案:

> "GQA（Grouped Query Attention）让多个 Q Head 共享一组 K/V Head。MiniMind 用 8 个 Q Head、2 个 KV Head，每 4 个 Q Head 共享 1 组 K/V。
>
> 与 MHA（Multi-Head Attention）相比： - MHA: Q/K/V head 数量相同（8/8/8） - GQA: Q head 多，K/V head 少（8/2/2） - MQA: 极端情况（8/1/1）
>
> GQA 的好处是减少 K/V 的参数和计算量，特别是推理时 KV-Cache 更小。效果上接近 MHA，但显存和速度更优。"

Q6: 为什么用 RMSNorm 而不是 LayerNorm?

标准答案:

> "RMSNorm 去掉了 LayerNorm 中的均值中心化（减去 mean）步骤，只保留了方差归一化（除以 RMS）。
>
> 优势： 1. 计算更快（少一次 mean 统计） 2. 实践中效果与 LayerNorm 相当 3. LLaMA 系列模型验证了 RMSNorm 在大模型上的有效性
>
> RMSNorm 公式：$y = x / RMS(x) \times \gamma$，其中 $RMS(x) = \sqrt{(mean(x^2))}$"

Q7: SwiGLU 是什么?

标准答案:

> "SwiGLU 是 FFN 中的激活函数，结合了 Swish 激活和 GLU（Gated Linear Unit）门控机制。
>
> 公式：$SwiGLU(x) = Swish(xW_1) \odot (xW_2)$，然后再过 W_3 投影回去。
>
> 其中 $Swish(x) = x \times \sigma(x)$。相比 ReLU，SwiGLU 效果更好但参数多 50%（3 个矩阵 vs 2 个矩阵）。LLaMA 等主流模型都采用 SwiGLU。"

Q8: 预训练和 SFT 的区别?

标准答案:

> "预训练和 SFT 的核心区别：
>
> **预训练**： - 目标：学习语言知识和世界知识 - 数据：大量无标注文本（维基百科、网页等） - 训练方式：Next Token Prediction，所有 token 都计算 loss - 数据量：3B+ tokens

> **SFT（有监督微调）**： - 目标：学会按照指令格式回答 - 数据：指令-回答对（prompt-response） - 训练方式：只对 response 部分计算 loss（Loss Mask） - 数据量：相对较小（几十万条）"

Q9: Loss Mask 是什么?

标准答案:

> "Loss Mask 是 SFT 训练中的关键技术。在一条训练数据中，我们不希望模型去'预测' prompt 部分（这是给定的输入），只希望模型学习生成 response 部分。
>
> 实现：构建一个 mask 矩阵，prompt 位置为 0，response 位置为 1。计算 loss 时乘以这个 mask，prompt 部分的 loss 被置零。
>
> `输入: [user]你好[/user][assistant]你好！很高兴认识你[/assistant] Mask: [0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0]`
>
> 对于多轮对话，每轮的 user 部分 mask=0，assistant 部分 mask=1。"

Q10: LoRA 原理?

标准答案:

> "LoRA（Low-Rank Adaptation）是一种参数高效的微调方法。
>
> 核心思想：不修改原始权重 W，而是学习一个低秩增量 ΔW = BA，其中 B∈R^{d×r}，A∈R^{r×d}，r << d。
>
> 前向传播变成：y = (W + ΔW)x = Wx + BAx。
>
> 优势： 1. 可训练参数大幅减少（比如 r=16 时只有原来的 1%） 2. 原始模型参数冻结，不会遗忘 3. 可以合并回原权重，推理时无额外开销
>
> MiniMind 使用 LoRA(r=16) 进行微调，通常只对 Q/K/V/O 投影矩阵加 LoRA。"

Q11: DPO 和 PPO 的区别?

标准答案:

> "核心区别有四点： 1. **是否需要 RM**：DPO 不需要，PPO 需要 2. **数据类型**：DPO 用离线偏好对（Off-Policy），PPO 用在线采样（On-Policy） 3. **训练复杂度**：DPO 简单（2 个模型），PPO 复杂（4 个模型） 4. **探索能力**：DPO 不探索，PPO 可以探索新策略
>
> DPO 更适合偏好对齐（如安全性），PPO/GRPO 更适合能力提升（如推理能力）。"

Q12: 什么是 KV-Cache?

标准答案:

> "KV-Cache 是推理时缓存历史 token 的 Key 和 Value 向量的优化技术。
>
> 自回归生成每一步都需要和所有历史 token 做 Attention。如果不缓存，每步都要重新计算所有历史 K/V，计算量与序列长度的平方成正比。
>
> 用了 KV-Cache 后，每步只计算新 token 的 Q/K/V，K/V 追加到缓存中。单步计算量变成线性。
>
> 注意：训练时不用 KV-Cache，因为训练是并行处理整个序列，不存在重复计算问题。"

Q13: MoE 是怎么工作的?

标准答案:

> "MoE 用多个 FFN 专家替代单个 FFN，通过 Router 决定每个 token 送给哪些专家。
>
> MiniMind-3-MoE 配置：4 个专家，Top-1 路由。总参数 198M，每个 token 只激活 64M 参数（只过 1 个专家）。
>
> Router 就是一个线性层 + Softmax，输出每个专家的概率。选概率最高的 K 个专家处理该 token，加权求和得到输出。
>
> 训练时要加负载均衡 Loss，防止所有 token 涌向同一个专家（专家塌缩问题）。"

Q14: 训练用了什么数据?

标准答案:

> "分不同阶段使用不同数据： - **分词器**：使用中英文混合语料训练 BPE 分词器，词表 6400 - **预训练**：约 3B tokens 的中文文本，包括百科、问答等 - **SFT**：指令-回答对数据，支持单轮和多轮对话格式 - **DPO**：偏好对数据（dpo.jsonl），每条包含 chosen 和 rejected - **RL（PPO/GRPO）**：使用 SFT 数据的 prompt 部分，奖励由 InternLM2-1.8B-Reward 打分"

Q15: 训练了多久，用什么 GPU?

标准答案:

> "MiniMind 设计的核心理念就是低门槛。预训练在单张消费级 GPU（如 RTX 3090/4090）上可以在数小时内完成。SFT 和 DPO 更快，通常 1-2 小时。
>
> 64M 参数的模型非常小，单卡显存完全够用。如果使用 LoRA 微调，显存需求更低。这也是选择 64M 参数规模的原因——让个人开发者也能完整体验全流程。"

Q16: 模型有什么局限性?

标准答案:

> "主要局限： 1. **参数量小**：64M 参数能力有限，复杂推理和长文本理解较弱 2. **词表小**：6400 的词表覆盖有限，对少见字和英文处理较差 3. **训练数据有限**：3B tokens 相比主流模型（万亿级）差距很大 4. **幻觉严重**：小模型容易"编造"事实 5. **不支持多模态**：纯文本模型
>
> 但作为学习项目，这些局限是有意为之——目标是学习原理而非追求效果。"

Q17: 幻觉问题怎么缓解?

标准答案:

> "幻觉（Hallucination）指模型生成看似合理但实际错误的内容。缓解方法：
>
> 1. **更多高质量训练数据**：减少噪声数据，增加事实性数据
> 2. **RLHF/DPO 对齐**：让模型学会说"我不确定"
> 3. **RAG（检索增强生成）**：从外部知识库检索事实，作为上下文注入
> 4. **Temperature 调低**：减少采样随机性
> 5. **Self-consistency（自一致性）**：多次生成取共识
> 6. **Adaptive Thinking**：让模型先思考再回答
>
> 对于 MiniMind 这种小模型，幻觉是不可避免的，主要通过 RAG 和对齐来缓解。"

Q18: 为什么词表只有 6400?

标准答案:

> "6400 的小词表是有意设计的：
>
> 1. **Embedding 参数量控制**：Embedding 参数 = vocab_size × dim。如果用 32000 的词表，光 Embedding 就 16M 参数，占总参数的 25%
> 2. **训练效率**：小词表意味着 LM Head 的 softmax 更快
> 3. **学习目的**：展示分词器和词表设计的权衡
>
> 代价是分词粒度较粗，一些中文字可能需要多个 token 表示，文本压缩率不如大词表高。"

Q19: 权重共享（Weight Tying）是什么?

标准答案:

> "权重共享指 Embedding 层和 LM Head（输出层）共享同一组权重。
>
> Embedding: token_id → vector（查表），权重矩阵 $E \in R^{V \times D}$ LM Head: vector → logits（投影），权重矩阵 $W \in R^{D \times V}$

> 设 W = E^T，这就是权重共享。好处： 1. 减少一半的词表相关参数（对小模型影响显著） 2. 语义一致性：输入和输出的 token 表征在同一空间 3. 实践效果好：GPT-2、LLaMA 等都使用了权重共享"

Q20: 你在项目中遇到的最大挑战?

标准答案（示例，请根据自己的经历调整）：

> "最大的挑战是 RL 训练（PPO/GRPO）的调参。
>
> 小模型做 RL 训练有个特殊困难：模型本身生成能力有限，探索时产生的回答质量很低，奖励信号非常稀疏且噪声大。
>
> 我的解决过程： 1. 先确保 SFT 模型质量足够好（好的起点） 2. 用较小的学习率（防止模型崩溃） 3. 加大 KL 惩罚系数（不让模型跑太远） 4. 用 GRPO 替代 PPO（减少 Critic 网络的不稳定性）
>
> 这个过程让我深刻理解了 RL 训练中'探索与利用'的平衡。"

四、深挖追问应对策略

4.1 面试官的追问模式

面试官追问通常有三种模式：

纵向深挖：在一个点上不断深入

```
你用了 RoPE → RoPE 的数学推导 → 为什么用 10000 作为 base → 修改 base 会怎样
```

横向拓展：从一个点出发联系其他知识

```
你用了 GQA → 和 MHA 区别 → 和 MQA 的区别 → 对 KV-Cache 有什么影响
```

挑战质疑：质疑你的选择或理解

```
你说 DPO 好 → 那为什么 OpenAI 用 PPO 不用 DPO → DPO 有什么局限
```

4.2 应对策略

原则一：承认不知道比胡编强

如果面试官问到你不清楚的知识点，坦诚说"这个我不太了解，但我的理解是..."比编造一个错误答案好得多。

原则二：从已知出发推理

```
面试官：RoPE 的 base 从 10000 改成 100000 会怎样？
你：（我不确定具体效果，但可以推理）
→ base 越大，低维的频率越低，旋转越慢
→ 这意味着位置编码的周期变长
→ 可能有助于处理更长的序列
→ 但可能降低短距离的位置精度
```

原则三：引导到自己擅长的方向

```
面试官：说说你对分布式训练的理解？
你：我在 MiniMind 中主要用了 DDP 做数据并行，因为模型只有 64M 参数，
    单卡完全放得下。不过我了解模型并行的基本概念...
    （然后主动转向你更熟悉的话题）
    ...我在 MoE 架构中思考过专家并行的可能性...
```

4.3 高频深挖点的准备

以下是面试官最喜欢深挖的几个方向，提前准备好：

Attention 相关： - Attention 复杂度是多少？$O(n^2d)$ - 怎么降低复杂度？FlashAttention（IO 优化）、稀疏 Attention、线性 Attention - Causal Mask 是什么？防止当前 token 看到未来的信息

位置编码： - ALiBi vs RoPE？ALiBi 是 Attention Bias，RoPE 是旋转编码 - 绝对位置编码的问题？不能外推到更长序列 - NTK-aware RoPE 和 YaRN 的区别？

训练相关： - 梯度累积是什么？累积多个 mini-batch 的梯度再更新，等效于更大 batch - 混合精度训练？FP16 前向/反向 + FP32 权重更新 - 学习率 warmup 的作用？初始阶段 lr 太大会导致训练不稳定

对齐相关： - RLHF 的完整流程？SFT → 训练 RM → PPO - Constitutional AI 是什么？用规则约束代替人类反馈 - DPO 的变体？IPO、KTO、ORPO 等

五、加分项

5.1 如果你做了额外工作

以下行为会在面试中加分：

1. 微调私有数据

> "我用公司/学校的 [领域] 数据对 MiniMind 做了 SFT，效果是 [具体指标]"

2. 对比实验

> "我对比了 RoPE 和学习式位置编码的效果，发现在短序列上差别不大，但长序列外推 RoPE 明显更好"

3. 训练 MoE 版本

> "我从 Dense 版本扩展到了 MoE，遇到了专家塌缩问题，通过调整负载均衡系数解决了"

4. 做了消融实验

> "我做了 GQA vs MHA 的消融实验，在 64M 参数下 GQA 可以节省 20% 的 KV-Cache 显存而精度损失小于 1%"

5. 代码优化

> "我用 FlashAttention 替换了标准 Attention 实现，推理速度提升了 2 倍"

5.2 展示学习深度的加分话术

关联最新论文：

> "我注意到 DeepSeek-V2 也用了 MoE 架构，但他们用了 DeepSeekMoE 的共享专家机制，这和 MiniMind 的标准 MoE 有区别..."

理解设计权衡：

> "MiniMind 选择 6400 的小词表是为了控制 Embedding 参数占比，但如果要做英文任务，可能需要扩展到至少 32K..."

思考改进方向：

> "如果要提升 MiniMind 的效果，我认为最重要的三个方向是：(1) 扩大训练数据；(2) 增大词表；(3) 在 SFT 阶段引入更高质量的数据。"

六、面试心态和技巧

6.1 心态准备

1. **项目是载体，能力是目的**：面试官考察的是你的技术理解力，不是 MiniMind 项目本身
2. **不是所有问题都有标准答案**：开放性问题展示你的思考过程更重要
3. **做了就是做了，没做就没做**：不要夸大自己的贡献

6.2 实用技巧

技巧一：准备纸笔，画图讲解

面试时如果需要解释 Attention 机制或 MoE 路由，画一个简单的图会比口头描述清晰 10 倍。

```
Attention 计算流程图:
Input → [Q] [K] [V] → Q×K^T → Scale → Mask → Softmax → ×V → Output

MoE 流程图:
Input → Router → [Expert 1, Expert 2, Expert 3, Expert 4] → Weighted Sum → Output
                    ↑ Top-1 选择
```

技巧二：主动总结

回答完一个复杂问题后，用一句话总结：

> "简单来说，KV-Cache 就是空间换时间——缓存历史的 K/V，避免推理时重复计算。"

技巧三：反问环节准备好问题

```
- 你们团队目前在做什么方向的大模型工作？
- 你们的模型训练用什么规模的数据和算力？
- 你们是否有做 RLHF/DPO 相关的工作？
```

6.3 常见陷阱

陷阱一：只会背诵，不能灵活应用

面试官可能会变换问法： - "如果你的模型 loss 不下降怎么排查？" - "如果要把模型从 64M 扩展到 1B，需要改什么？"

→ 需要真正理解原理，不能只背答案

陷阱二：过度关注细节，忽略全局

如果面试官问"介绍一下你的项目"，不要一上来就讲 RoPE 的数学推导。先给全景，再根据面试官兴趣深入。

陷阱三：把项目说成"跑了一下开源代码"

要强调你**理解了什么**、**思考了什么**、**尝试了什么**，而不是"我 clone 了代码，改了几个参数，跑了一下"。

🎤 面试考点（综合回顾）

Q1: 一句话概括 MiniMind 项目?

答：MiniMind 是一个从零实现的 64M 参数大语言模型，涵盖了 Transformer 架构、预训练、SFT、DPO/PPO 对齐、MoE 扩展和多框架部署的完整技术栈。

Q2: 预训练 → SFT → RLHF 三阶段各自的作用?

答： - 预训练：学习语言知识和世界知识（"学会说话"） - SFT：学习指令跟随和对话格式（"学会回答"） - RLHF/DPO：对齐人类偏好和价值观（"学会好好说话"）

Q3: 如果你只能做一个改进来提升 MiniMind 效果，你会做什么？

答： （开放题，展示思考能力）

> "我会扩大训练数据量。当前 3B tokens 的预训练数据相对不足，研究表明数据量对小模型的影响甚至比参数量更大（参考 Chinchilla scaling law）。如果能用 10-30B tokens 的高质量数据，效果应该会有显著提升。"

Q4: Transformer 的计算瓶颈在哪？

答： - 训练时：Attention 的 $O(n^2)$ 复杂度是瓶颈，长序列训练很慢 - 推理时：自回归生成是 memory-bound（受 GPU 显存带宽限制），不是 compute-bound - KV-Cache 的显存占用也是推理的瓶颈

Q5: 你怎么评估模型效果？

答： - 定量：用 perplexity（困惑度）衡量语言建模能力，用下游任务 benchmark（如 C-Eval）评估知识 - 定性：人工评估对话质量、连贯性、准确性 - 对齐效果：通过 A/B 测试或人类偏好评分

✅ 自测题

1. **模拟面试**：请用 1 分钟介绍 MiniMind 项目（计时练习）。
2. **手写**：在纸上画出 Transformer Decoder Block 的结构图。
3. **快问快答**：不看笔记，回答以下问题：
4. RoPE 的全称？
5. GQA 和 MHA 的区别？
6. DPO 的损失函数中 β 的作用？
7. KV-Cache 为什么只缓存 K 和 V？
8. MoE 的负载均衡 Loss 解决什么问题？
9. **深挖演练**：让朋友/同学扮演面试官，从 Top 20 问题中随机挑选 5 个追问。
10. **简历打磨**：根据本节提供的模板，写出你自己的简历项目描述。

> **▼ 参考答案（快问快答部分）**

> - RoPE: Rotary Position Embedding（旋转位置编码） - GQA 让多个 Q Head 共享一组 K/V Head；MHA 中 Q/K/V head 数量相同 - β 控制策略模型偏离参考模型的程度，越大越保守 - 因为 Q 每次不同（对应当前新 token），而 K/V 历史部分不变可以复用 - 防止专家塌缩（所有 token 涌向同一个专家）

课程总结

恭喜你完成了 MiniMind 学习教程的全部 22 节课程！

回顾我们的学习路径：

```
Phase 1: 基础知识 (L01-L06)
  Tokenizer → Embedding → RoPE → Attention/GQA → FFN/SwiGLU → Transformer Block

Phase 2: 训练流程 (L07-L12)
  预训练 → Loss Mask → SFT → LoRA → 数据工程 → 评估

Phase 3: 理论深化 (L13-L16)
  Scaling Laws → 注意力变体 → 归一化 → 优化器

Phase 4: 高级特性与面试通关 (L17-L22)
  DPO → PPO/GRPO → MoE → 推理优化 → 部署 → 面试通关
```

你现在拥有了完整的 LLM 知识体系。下一步：**动手实践，反复迭代，把知识变成真正的能力。**

祝你面试顺利，拿到心仪的 Offer！